COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	**CPE 010 - Data Structures and Algorithms**	**SECTION:**	**CPE21S2**
DATE PERFORMED:	**07-24-26**	**DATE SUBMITTED:**	**07-29-26**
NAME:	**Christian Lee C. Bandigan**	**INSTRUCTOR:**	**Engr. Jimlord Quejado**

ACTIVITY NO.3 **Hands-on Activity 3.1 Linked Lists**

1. PROCEDURE OUTPUT

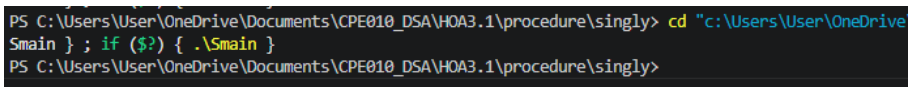
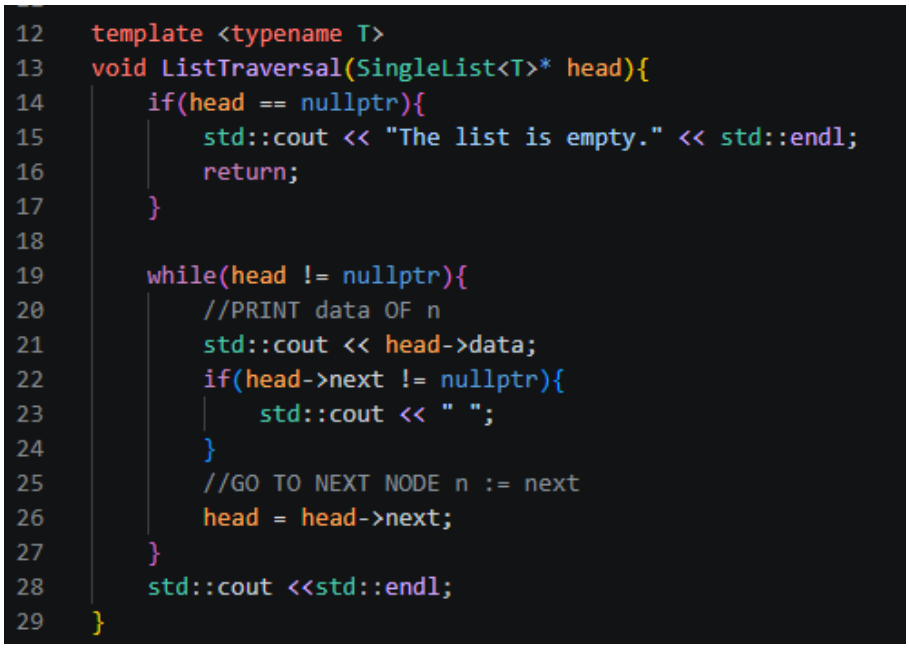
Screenshot	
Discussion	The simple implementation outputs nothing because we haven't printed anything yet. To improve the code we can add a function that traverses the list and prints the data.

Table 3-1. Output of Initial/Simple Implementation

Operation	**Screenshot**
Traversal	

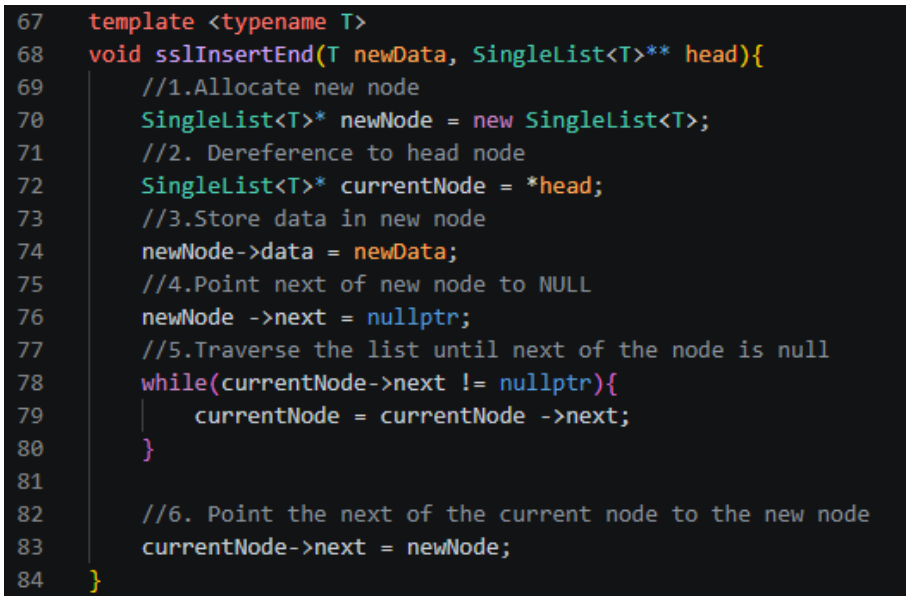
```
12  template <typename T>
13  void ListTraversal(SingleList<T>* head){
14      if(head == nullptr){
15          std::cout << "The list is empty." << std::endl;
16          return;
17      }
18
19      while(head != nullptr){
20          //PRINT data OF n
21          std::cout << head->data;
22          if(head->next != nullptr){
23              std::cout << " ";
24          }
25          //GO TO NEXT NODE n := next
26          head = head->next;
27      }
28      std::cout <<std::endl;
29  }
```

Insertion at head

```
31  template <typename T>
32  void sllinsertHead(T newData, SingleList<T>** currentHead){
33      //1. Allocate memory for new node
34      SingleList<T>*  newNode = new SingleList<T>;
35
36      //2.Put our data in new node
37      newNode->data = newData;
38
39      //3.Set Next of the new node to point to the previous Head
40      newNode -> next= *currentHead;
41
42      //4. Reset Head to point to the new node
43      *currentHead = newNode;
44  }
```

Insertion at any part of the list

```
46  template <typename T>
47  void sslGeneralInsert(T newData, SingleList<T>* preNode){
48      //1.Check if it is the head node (previous node is null)
49      if(preNode == nullptr){
50          //2.If null, print "Previous node cannot be null."
51          std::cout << "Previous value cannot be null" << std::endl;
52          return;
53      }
54      //3.Allocate a new node
55      SingleList<T>* newNode = new SingleList<T>;
56
57      //4.Store data in the new node
58      newNode->data = newData;
59
60      //5.Point new node to the node previous node was pointing to
61      newNode->next = preNode->next;
62
63      //6.Point previous node to the new node
64      preNode->next = newNode;
65  }
```

Insertion at the end	
Deletion of a node	

```
67  template <typename T>
68  void sslInsertEnd(T newData, SingleList<T>** head){
69      //1.Allocate new node
70      SingleList<T>* newNode = new SingleList<T>;
71      //2. Dereference to head node
72      SingleList<T>* currentNode = *head;
73      //3.Store data in new node
74      newNode->data = newData;
75      //4.Point next of new node to NULL
76      newNode ->next = nullptr;
77      //5.Traverse the list until next of the node is null
78      while(currentNode->next != nullptr){
79          currentNode = currentNode ->next;
80      }
81
82      //6. Point the next of the current node to the new node
83      currentNode->next = newNode;
84  }
```

```
86  template <typename T>
87  void sslDeleteNode(T findData, SingleList<T>** head){
88      //if there is nothing to be deleted?
89      if(*head == nullptr) return;
90
91      SingleList<T>* currNode = *head;
92      SingleList<T>* preNode = nullptr;
93
94      //searching the node: 1. the node doesen't exist + we found the data
95      while(currNode != nullptr && currNode ->data != findData){
96          preNode = currNode;
97          currNode = currNode ->next;
98      }
99      //if the data is not found currNode == nullptr in the while loop
100     if(currNode == nullptr){
101         std::cout << "The data is not found"<< std::endl;
102         return;
103     }
104
105     //if data was found
106     if(preNode == nullptr){
107         *head = currNode->next;
108     }else{
109         preNode->next = currNode ->next;
110     }
111
112     delete currNode;
113 }
```

Table 3-2. Code for the List Operations

a.	Source Code	`std::cout <<"Testing of Traversal: " <<std::endl;` `ListTraversal(head);`
	Console	`Testing of Traversal:` `C P E 0 1 0`

b.	Source Code	`std::cout << "Testing of sslinsertHead: " << std::endl;`
		`sllinsertHead('G',&head);`
		`ListTraversal(head);`

	Console	`Testing of sslinsertHead:` `G C P E 0 1 0`

c.	Source Code	`std::cout << "Testing of sslGeneralInsert: " << std::endl;`
		`sslGeneralInsert('E',head->next->next);`
		`ListTraversal(head);`

	Console	`Testing of sslGeneralInsert:` `G C P E E 0 1 0`

d.	Source Code	`std::cout << "Testing of deleteing the node: " << std::endl;`
		`sslDeleteNode('C', &head);`
		`ListTraversal(head);`

	Console	`Testing of deleteing the node:` `G P E E 0 1 0`
e.	Source Code	`sslDeleteNode('P', &head);` `ListTraversal(head);`

	Console	`Testing of deleteing the node:`
		`G P E E 0 1 0`
		`G E E 0 1 0`

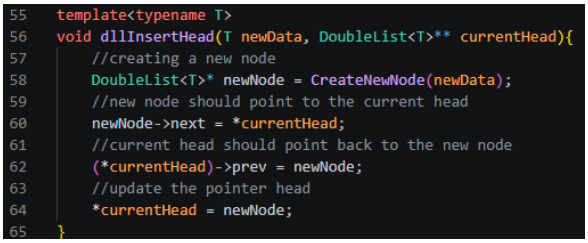
f.	Source Code	`std::cout << "Testing of deleteing the node: " << std::endl;`
		`sslDeleteNode('C', &head);`
		`ListTraversal(head);`
		`sslDeleteNode('P', &head);`
		`ListTraversal(head);`

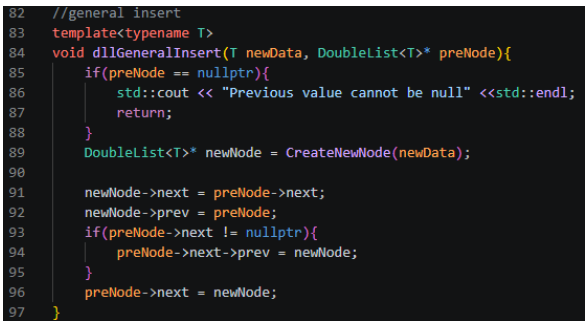
	Console	`Testing of deleteing the node:`
		`G P E E 0 1 0`
		`G E E 0 1 0`

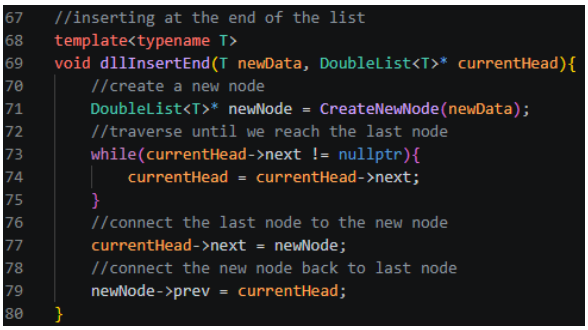
Table 3-3. Code and Analysis for Singly Linked Lists

Screenshot(s)	Analysis


``
template<typename T>
void dllTraversal(DoubleList<T>* currentNode){
    //temp variable to store the last node visited
    DoubleList<T> *tail;
    //check if the list is empty
    if(currentNode == nullptr){
        std::cout << "The list is empty" <<std::endl;
        return;
    }
    std::cout << "Forwarding" << std::endl;
    //move forward
    while(currentNode != nullptr){
        std::cout << currentNode->data <<" ";
        tail = currentNode;
        currentNode = currentNode->next;
    }
    //add a next line
    std::cout << std::endl;
    std::cout << "Backwarding" << std::endl;
    //move backward
    while(tail != nullptr){
        std::cout << tail->data<<" ";
        tail = tail->prev;
    }
}
``
	In traversing a doubly linked list we can print it in two ways to show it is doubly by printing it forwards and backwards. The screenshot shows that the first step is to create a variable that will store the last node visited so we can print the list backwards by using prev. So, to print the doubly linked list we use two while loops, one that uses next and the other one uses prev.


``
template<typename T>
void dllInsertHead(T newData, DoubleList<T>** currentHead){
    //creating a new node
    DoubleList<T>* newNode = CreateNewNode(newData);
    //new node should point to the current head
    newNode->next = *currentHead;
    //current head should point back to the new node
    (*currentHead)->prev = newNode;
    //update the pointer head
    *currentHead = newNode;
}
``
	To insert a new node at the head is basically the same thing as sll but you have to assign the previous pointer of the previous head to the new head.

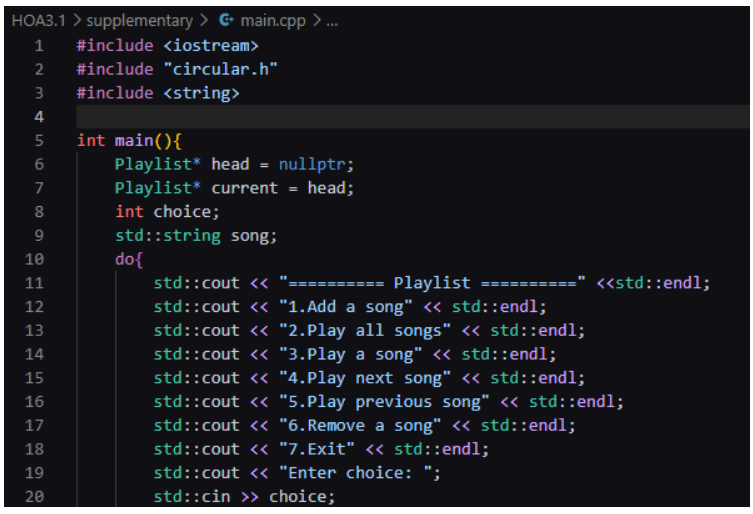

``
//general insert
template<typename T>
void dllGeneralInsert(T newData, DoubleList<T>* preNode){
    if(preNode == nullptr){
        std::cout << "Previous value cannot be null" <<std::endl;
        return;
    }
    DoubleList<T>* newNode = CreateNewNode(newData);
 	
    newNode->next = preNode->next;
    newNode->prev = preNode;
    if(preNode->next != nullptr){
        preNode->next->prev = newNode;
    }
    preNode->next = newNode;
}
``
	For general insert first create a new node, link it to the next node, set its previous pointer to the given node, update the next node's previous pointer, and finally connect the previous node to the new node.


``
//inserting at the end of the list
template<typename T>
void dllInsertEnd(T newData, DoubleList<T>* currentHead){
    //create a new node
    DoubleList<T>* newNode = CreateNewNode(newData);
    //traverse until we reach the last node
    while(currentHead->next != nullptr){
        currentHead = currentHead->next;
    }
    //connect the last node to the new node
    currentHead->next = newNode;
    //connect the new node back to last node
    newNode->prev = currentHead;
}
``
	To insert a new node at the end, create a new node, traverse the list until the last node, connect the last node to the new node, and lastly set the new node's previous pointer to the last node.


``
//deleting a node
template<typename T>
void dllDeleteNode(T findData, DoubleList<T>** head){
    if(*head == nullptr)return;
 	
    DoubleList<T>* currNode = *head;
    DoubleList<T>* preNode = nullptr;
 	
    while(currNode != nullptr && currNode->data !=findData){
        preNode = currNode;
        currNode = currNode->next;
    }
    if(currNode == nullptr){
        std::cout << "Data is not found." << std::endl;
        return;
    }
    if(preNode == nullptr){
        *head = currNode->next;
        if(*head != nullptr){
            (*head)->prev = nullptr;
        }
    }else{
        preNode->next = currNode->next;
        if(currNode->next !=nullptr){
            (currNode->next)->prev = preNode;
        }
    }
    delete currNode;
}
``
	To delete a node I used a while loop to search for the node with the given data and then updated the links of the surrounding nodes to remove it from the list. Moreover I need to adjust the head if the first node is deleted, and finally delete the current node.

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY



HOA3.1 > supplementary > main.cpp > ...

```
#include <iostream>
#include "circular.h"
#include <string>

int main(){
    Playlist* head = nullptr;
    Playlist* current = head;
    int choice;
    std::string song;
    do{
        std::cout << "========== Playlist ==========" <<std::endl;
        std::cout << "1.Add a song" << std::endl;
        std::cout << "2.Play all songs" << std::endl;
        std::cout << "3.Play a song" << std::endl;
        std::cout << "4.Play next song" << std::endl;
        std::cout << "5.Play previous song" << std::endl;
        std::cout << "6.Remove a song" << std::endl;
        std::cout << "7.Exit" << std::endl;
        std::cout << "Enter choice: ";
        std::cin >> choice;
```

```
22          switch(choice){
23              case 1:
24                  std::cout << "Enter new song: ";
25                  std::cin >> song;
26                  addSong(song,&head);
27                  std::cout << "Successfully added the song.\n" << std::endl;
28                  break;
29              case 2:
30                  playSongs(head);
31                  break;
32              case 3:
33                  std::cout << "Enter song you want to play: ";
34                  std::cin >> song;
35                  playSong(song,&current,&head);
36                  break;
37              case 4:
38                  nextSong(&current);
39                  break;
40              case 5:
41                  prevSong(&current);
42                  break;
43              case 6:
44                  std::cout << "Enter the song you want to remove: ";
45                  std::cin >> song;
46                  removeSong(song,&head);
47                  std::cout << "Successfully removed the song.\n" <<std::endl;
48                  break;
49              case 7:
50                  std::cout << "Goodbye." <<std::endl;
51                  break;
52              default:
53                  std::cout << "Invalid Input. Please Try Again." <<std::endl;
54                  break;
55          }
56      }while(choice != 7);
57      return 0;
58  }
```

Figure 1. Main

HOA3.1 > supplementary > C circular.h > playSong(std::string, Playlist **, Playlist **)

```cpp
#ifndef CIRCULAR_H
#define CIRCULAR_H
#include <iostream>
#include <string>

class Playlist{
    public:
        std::string song;
        Playlist* next = nullptr;
        Playlist* prev = nullptr;
};

Playlist *NewSong(std::string newSong){
    Playlist *newNode = new Playlist;
    newNode->song = newSong;
    newNode->next = nullptr;
    newNode->prev = nullptr;

    return newNode;
}
```

```cpp
void addSong(std::string song, Playlist** currHead){
    Playlist* newSong = NewSong(song);
    if(*currHead == nullptr){
        *currHead = newSong;
        (*currHead)->next = newSong;
        (*currHead)->prev = newSong;
        return;
    }
    Playlist* tail = (*currHead)->prev;
    tail->next = newSong;
    newSong->next = *currHead;
    newSong->prev = tail;
    (*currHead)->prev = newSong;
}

void playSongs(Playlist* currSong){
    if(currSong == nullptr){
        std::cout <<"Playlist is empty." << std::endl;
        return;
    }
    Playlist *head = currSong;
    std::cout << "========== Playlist ==========" <<std::endl;
    do{
        std::cout << currSong->song;
        if(currSong->next != head){
            std::cout << " -> ";
        }
        currSong = currSong->next;
    }while(currSong != head);
    std::cout << std::endl;
}
```

```cpp
void playSong(std::string findSong, Playlist** currentSong, Playlist** head){
    if(*head == nullptr){
        std::cout << "Playlist is empty." << std::endl;
        return;
    }
    Playlist* currSong = *head;
    while(currSong->song != findSong){
        currSong = currSong->next;
        if(currSong == *head){
            std::cout << "Song is not found in the playlist." << std::endl;
            return;
        }
    }
    *currentSong = currSong;
    std::cout << "Now Playing:" << std::endl;
    std::cout << currSong->song <<std::endl;
}

void nextSong(Playlist** currentSong){
    if(currentSong == nullptr){
        std::cout << "Playlist is empty." << std::endl;
        return;
    }
    std::cout << "Now Playing:" <<std::endl;
    std::cout << (*currentSong)->next->song <<std::endl;
    *currentSong = (*currentSong)->next;
}

void prevSong(Playlist** currentSong){
    if(currentSong == nullptr){
        std::cout << "Playlist is empty." << std::endl;
        return;
    }
    std::cout << "Now Playing:" <<std::endl;
    std::cout << (*currentSong)->prev->song <<std::endl;
    *currentSong = (*currentSong)->prev;
}
```

```cpp
void removeSong(std::string findSong, Playlist** head){
    if(*head == nullptr){
        std::cout << "Playlist is empty." << std::endl;
        return;
    }
    Playlist* currSong = *head;

    while(currSong->song != findSong){
        currSong = currSong->next;
        if(currSong == *head){
            std::cout << "Song is not found in the playlist." << std::endl;
            return;
        }
    }
    if(currSong == *head){
        *head = currSong->next;
    }
    currSong->next->prev = currSong->prev;
    currSong->prev->next = currSong->next;
    delete currSong;
}

#endif
```

Figure 2. Header File

```
========== Playlist ==========
1.Add a song
2.Play all songs
3.Play a song
4.Play next song
5.Play previous song
6.Remove a song
7.Exit
Enter choice: 1
Enter new song: Lulay
Successfully added the song.

========== Playlist ==========
1.Add a song
2.Play all songs
3.Play a song
4.Play next song
5.Play previous song
6.Remove a song
7.Exit
Enter choice: 1
Enter new song: Kabisado
Successfully added the song.

========== Playlist ==========
1.Add a song
2.Play all songs
3.Play a song
4.Play next song
5.Play previous song
6.Remove a song
7.Exit
Enter choice: 1
Enter new song: UNDERSTAND
Successfully added the song.
```

Figure 3. Output for Adding a Song

```
========== Playlist ==========
1.Add a song
2.Play all songs
3.Play a song
4.Play next song
5.Play previous song
6.Remove a song
7.Exit
Enter choice: 2
========== Playlist ==========
Lulay -> Kabisado -> UNDERSTAND
```

Figure 4. Output of Play All Songs

```
========== Playlist ==========
1.Add a song
2.Play all songs
3.Play a song
4.Play next song
5.Play previous song
6.Remove a song
7.Exit
Enter choice: 3
Enter song you want to play: Kabisado
Now Playing:
Kabisado
```

Figure 5. Output for Play a Song

```
========== Playlist ==========
1.Add a song
2.Play all songs
3.Play a song
4.Play next song
5.Play previous song
6.Remove a song
7.Exit
Enter choice: 4
Now Playing:
UNDERSTAND
========== Playlist ==========
1.Add a song
2.Play all songs
3.Play a song
4.Play next song
5.Play previous song
6.Remove a song
7.Exit
Enter choice: 4
Now Playing:
Lulay
```

Figure 6. Output for Play Next Song

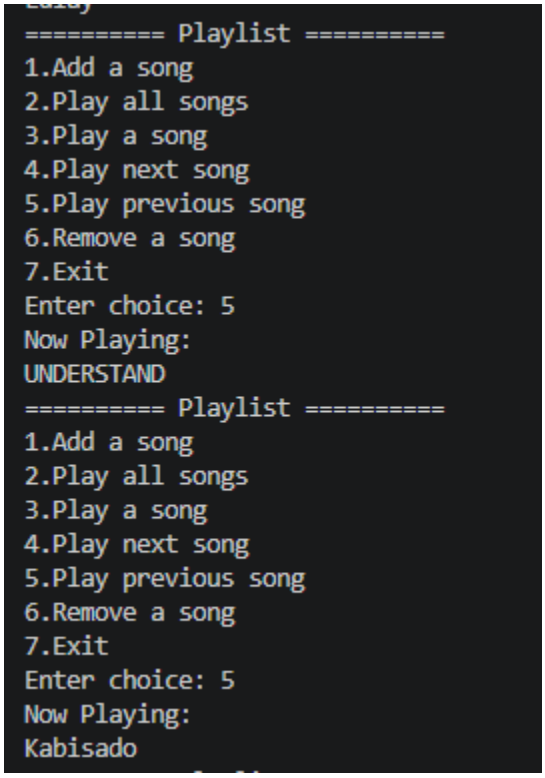


Figure 7. Output for Play Previous Song

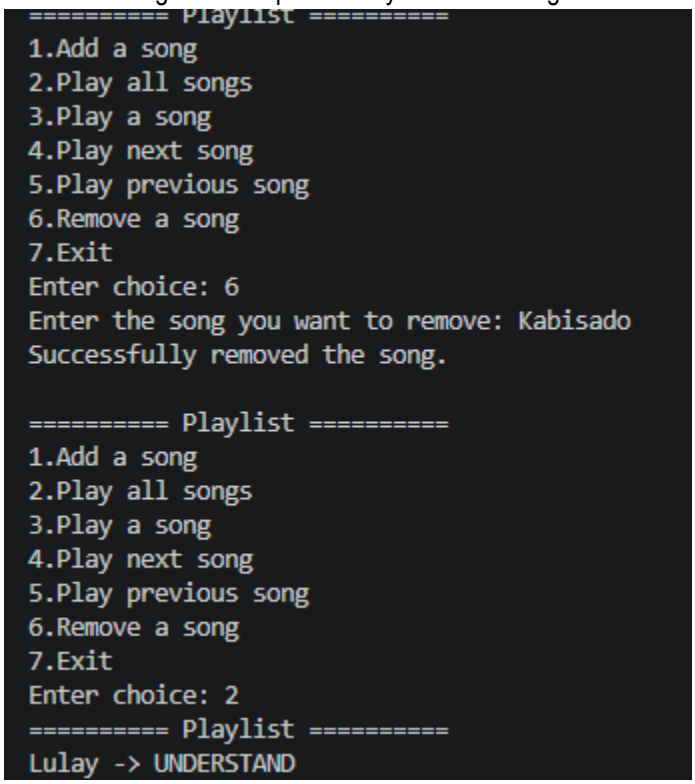


Figure 8. Output for Removing a Song

Analysis

To make a circular linked list, the class and node creation is basically the same but in the add song method the last next pointer should point to the head and not to null and the same goes for the first previous pointer should point to the tail.

To traverse a circular linked list to play all the songs, I first created a pointer and assigned it to the current head. I then used a do while loop to print out the songs that use the pointer as the condition. The do while loop stops when the current song is not equal to the head since this is a circular linked list.

To make a next and previous song function, I also created a play one song function and current song pointer to know the current song playing. Inside the three functions it of course prints the song and either increment or decrement the current pointer.

To remove a song from the playlist, I just modified the doubly linked list delete node function to make it work on a circular linked list

Conclusion

In this activity I learned what linked lists are and how to implement them. Linked Lists are a linear data structure and unlike arrays they are dynamic and use pointers. In this activity I implemented two types of linked list singly and doubly. A singly linked list only has a pointer to the next node or null while a doubly linked list has a pointer for both the next node and the previous. I also created some operations to handle these linked lists. These operations include creation, traversal, insertion at head and end, general insertion, removal of nodes. These operations for the two lists are mostly similar but you have to modify some parts. Moreover, in the supplementary activity I created a song playlist using circular linked list which is similar to a doubly linked list but instead of the head and tail pointing to null it points to each other.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.